TEAM 1 · MINI PROJECT IMPLEMENTATION

Arithmetic Expression Compiler

A C-based compiler front end

TEAM MEMBERS

Ayrin Anna Binu-71

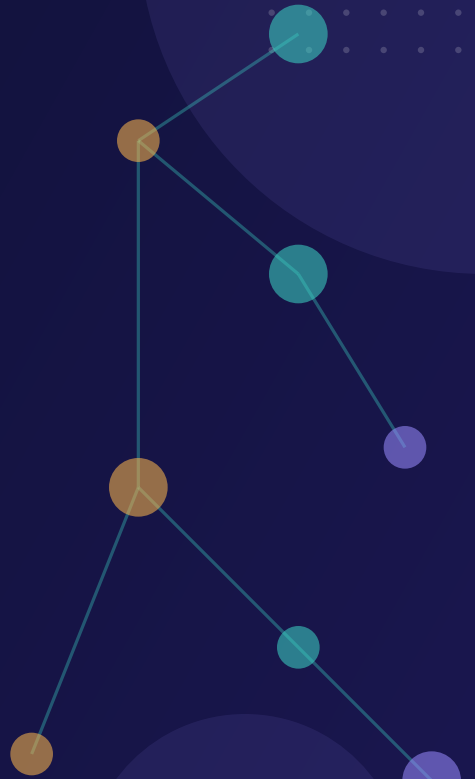
Angela Ann Byju-51

Ann Maria Mathew-54

Anns Thomas-58

Alwin Pramod-42

Albert Joseph-31



Core Objectives

Foundational goals for the compiler front-end.

Recognition

Lexical token identification.

Validation

Syntax precedence enforcement.

Construction

Abstract syntax tree generation.

Reporting

Malformed expression detection.

Pipeline Architecture

1

Input Stream

Raw character ingestion.

2

Lexical Analyzer

Token generation mapping.

3

Recursive Parser

Grammar validation logic.

4

AST Builder

Hierarchical tree representation.

5

Semantic Check

Zero-division anomaly detection.

Lexical Scanner Output

PHASE 1

TOKEN_NUM

Multi-digit numeric values.

TOKEN_ID

Variable identifiers.

TOKEN_OP

Mathematical operators.

TOKEN_EOF

End of expression marker.

Context-Free Grammar

```
E  →  T E′

E′  →  + T E′  |  - T E′  |  ε

T  →  F T′

T′  →  * F T′  |  / F T′  |  ε

F  →  ( E )  |  id  |  num  |  + F  |  - F
```

HIGHEST

Parentheses and unary signs.

MIDDLE

* and /

LOWEST

+ and -

Recursive Parser

PHASE 2

C FUNCTION MAPPING

- Top-down execution flow.
- Mutually recursive functions.
- Precise error localization.

NON-TERMINAL E

`parse_expression()`

NON-TERMINAL T

`parse_term()`

NON-TERMINAL F

`parse_factor()`

AST Construction

PHASE 3

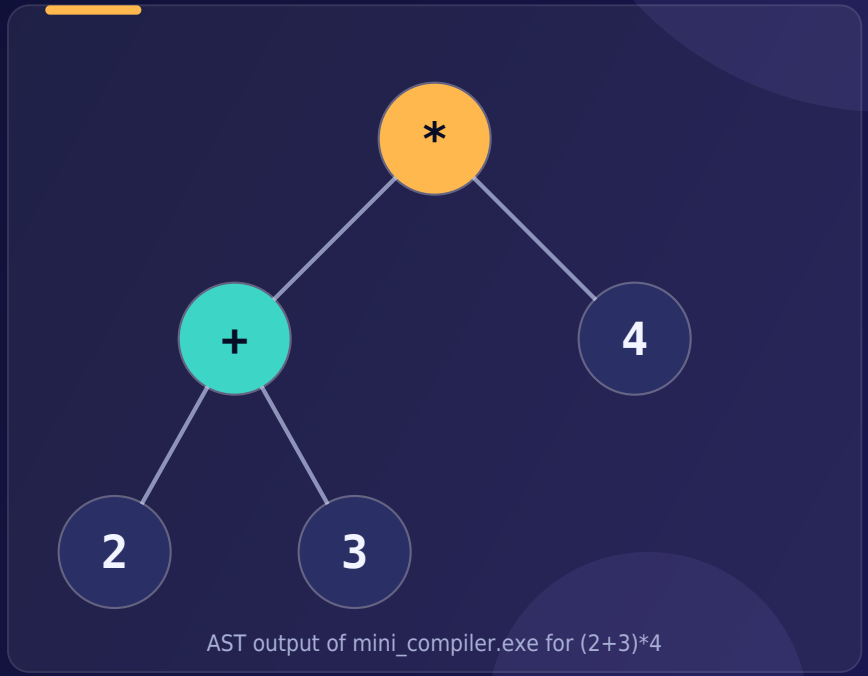
- Example input: (2+3)*4

- Root: *

- Left child: + with children 2 and 3

- Right child: number 4

- Parentheses force the addition to be grouped first. The tree preserves precedence; it does not evaluate the expression.



AST output of mini_compiler.exe for (2+3)*4

Syntax Validation

Verified with test runs of mini_compiler.exe.

Valid Expressions

- ✓ `2+3`
- ✓ `5-2*4`
- ✓ `3*4`
- ✓ `8/2`
- ✓ `(2+3)*4`

Invalid Syntax

- ✗ `2+`
  position 2: expected operand
- ✗ `*3`
  position 0: unexpected '*'
- ✗ `(2+3`
  position 4: expected ')'
- ✗ `2*/3`
  position 2: unexpected '/'
- ✗ `2 3`
  position 2: unexpected token

Semantic Check

Detects invalid operations prior to runtime.

`a/0`

THROWS ZERO-DIVISION WARNING

Error Resolution

Multi-layered validation across the compilation pipeline.

Lexical Rejection

Immediate invalidation of unrecognized symbols.

Syntax Validation

Detection of structural grammar violations.

Semantic Analysis

Post-AST traversal logic checks.

Current Boundaries

Current Boundaries

- No machine-code generation.
- Symbol table absence.
- No runtime expression evaluation.

Development Roadmap

- Richer semantic analysis.
- Symbol table integration.
- Intermediate code generation.

Team Contributions

Ann Maria Mathew

Integration core.

Anns Thomas

Lexical analysis.

Angela Ann Byju

Grammar parser.

Ayrin Anna Binu

Semantic checks.

Albert Joseph

Documentation architecture.

Alwin Pramod

Quality assurance.

Conclusion

- The Arithmetic Expression Compiler demonstrates the essential stages of a compiler front end.
- The lexical analyzer converts input expressions into tokens.
- The parser validates syntax, grammar, and operator precedence.
- The Abstract Syntax Tree (AST) represents the structure of arithmetic expressions.
- Semantic checks help identify errors and invalid operations.
- Overall, the project provides a clear understanding of compiler design and arithmetic expression validation.

